

# Automated Reinforcement Learning: Stability and Hyperparameter Sensitivity on CartPole-v1

A Comparative Study of Policy Gradient Methods

Purvika Panwar

`<p.panwar@stud.hs-schmalkalden.de>`

Matriculation No.: 319301

## Abstract

In reinforcement learning, reproducibility is a daily problem and not just a theoretical one. Training the same agent twice with the same code but slightly different settings or random seeds can produce learning curves that look completely different [1]. In this project we study this behaviour in a concrete and controlled setting. We implement a REINFORCE agent [2] and use Proximal Policy Optimization (PPO) from Stable-Baselines3 [3] on CartPole-v1, then run systematic experiments over multiple random seeds and several hyperparameters. Across our comparative runs, REINFORCE is extremely unstable, with large jumps and crashes, while PPO learns much more smoothly and is clearly more reliable. However, PPO is far from “plug and play”: changing the discount factor, learning rate, or clip range can completely change the outcome. For example, with a discount factor of  $\gamma = 0.90$  PPO often plateaus around 200 reward, whereas  $\gamma = 0.99$  solves the task in a stable way. Similar sensitivity appears for the learning rate and clip range. These results support the core claim from the AutoRL literature [4]: modern RL algorithms are powerful but fragile, and automated methods for choosing hyperparameters are not a luxury but a requirement.

**Keywords:** AutoRL, PPO, REINFORCE, Hyperparameter Tuning, Stability, Reproducibility

## 1 Introduction

Getting RL to work in practice is much more fragile than it looks in textbooks. On paper the goal is simple: find a policy that maximizes expected return. In reality, two runs of the same algorithm with different random seeds can give completely different stories. In our early REINFORCE experiments, one seed reached high reward quickly, another took hundreds of episodes, and a third seemed to collapse again after first improving. It was

not obvious whether this behaviour was caused by bugs or whether it is simply the nature of the algorithm.

Henderson et al. showed that this behaviour is not unusual but rather typical: different seeds and hyperparameters can easily change the ranking between RL algorithms, which makes results hard to trust [1]. At the same time, there is now a growing AutoRL literature that argues for automating the configuration of RL agents instead of hand-tuning them [4]. Before building such an automated system it is useful to have a small but clear test-bed that shows where things actually break.

The goal of this project is to build such a test-bed on CartPole-v1 and to answer three questions:

1. How do REINFORCE and PPO compare in terms of learning speed and stability across random seeds?
2. How sensitive is PPO to key hyperparameters such as the discount factor, learning rate, and clip range?
3. Do the results support the argument that AutoRL is needed even for a simple benchmark like CartPole?

Instead of reporting single runs, we try to follow the recommendations by Henderson et al. and always look at multiple seeds and simple uncertainty estimates where possible [1]. Concretely, we compute the mean and standard deviation across seeds for each episode index and visualize this as a solid mean curve with a shaded band. This already makes the experiments feel more honest and also much easier to interpret.

## 2 Background

### 2.1 Markov Decision Processes and Return

CartPole-v1 is modelled as a Markov Decision Process with state space  $\mathcal{S}$ , action space  $\mathcal{A}$ , transition dynamics  $P$ , reward function  $R$ , and discount factor  $\gamma$  [5]. At each timestep  $t$  the agent observes a state  $s_t$ , chooses an action  $a_t$ , receives a reward  $r_t$ , and moves to the next state. The objective is to maximize the expected discounted return

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right]. \quad (1)$$

This formula comes directly from the standard definition of return in reinforcement learning [5]. It tells us how much total reward we expect if we start with policy parameters  $\theta$  and then follow the policy forever. In our project this objective is the quantity that all algorithms implicitly try to maximize, even though we never compute  $J(\theta)$  exactly. Instead, we approximate its gradient from sampled episodes.

The discount factor  $\gamma$  controls how far into the future the agent “cares”. Lower values make the agent myopic and focus on immediate rewards, while higher values emphasize long-term returns. In theory, CartPole episodes are short (maximum length 500), so one

might expect a wide range of  $\gamma$  values to work. In practice, our experiments below show that the choice of  $\gamma$  can make the difference between success and failure and is one of the most sensitive hyperparameters.

## 2.2 REINFORCE

REINFORCE is one of the classic policy gradient methods [2]. The idea is conceptually simple: run entire episodes, compute the discounted return  $G_t$  from each timestep, and then update the policy parameters in the direction that increases the log-probability of actions that led to high returns. The estimator we use is

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) G_t^i. \quad (2)$$

This formula is derived from the policy gradient theorem combined with a Monte-Carlo estimate of the return [5]. Intuitively, if an action  $a_t$  in state  $s_t$  leads to a high future return  $G_t$ , we want to increase the log-probability of taking this action again. If the return is low, we want to decrease it. In our project, this is the main update rule used by the REINFORCE agent.

The estimator is unbiased but has very high variance because each  $G_t$  depends on an entire random trajectory. In our implementation this shows up as very jagged learning curves. A single unlucky episode with low reward can produce a large negative gradient and undo much of the earlier progress. This is consistent with the standard criticism of REINFORCE and explains why more advanced methods are now preferred [5].

## 2.3 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a modern policy gradient method that tries to keep the good parts of policy gradients but control the update size [3]. The clipped objective we use is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (3)$$

where  $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$  is the probability ratio and  $\hat{A}_t$  is an advantage estimate. This loss function is taken directly from the original PPO paper [3].

The inner minimum has an important interpretation. The term  $r_t(\theta) \hat{A}_t$  is the standard policy gradient objective: if the advantage is positive we want to increase the probability of the action, and if it is negative we want to decrease it. The clipped term restricts  $r_t(\theta)$  to the interval  $[1 - \epsilon, 1 + \epsilon]$ . If a proposed update would change the policy too aggressively, the clipped objective is smaller and therefore preferred by the minimum. In practice this acts like a safety belt and produces much smoother learning behaviour.

In our experiments we do not re-implement PPO from scratch but use the implementation from Stable-Baselines3, which follows the original algorithm closely and is widely used in practice. This allows us to focus on stability and hyperparameters instead of low-level debugging.

## 3 Methods

### 3.1 Environment and Agents

All experiments are run on CartPole-v1 from OpenAI Gym. An episode starts with the pole roughly upright and ends if the angle becomes too large or the cart goes out of bounds. The agent receives a reward of 1 per timestep and the maximum episode length is 500, so a perfectly balanced policy gets a return of 500 while a random agent typically gets something around 20–25.

We compare three agents:

- **Random policy:** uniformly samples left or right. This is a simple baseline to show how “no learning” looks.
- **REINFORCE:** our own PyTorch implementation with an MLP policy (two hidden layers, 128 units, ReLU). The learning rate is set to  $10^{-3}$  and discount factor to 0.99.
- **PPO:** Stable-Baselines3 PPO with the default MLP policy and default hyperparameters unless otherwise stated. This includes a clip range of 0.2 and discount factor  $\gamma = 0.99$ .

For PPO all experiments use the same training budget of 50 000 timesteps, which corresponds to roughly 300–500 episodes depending on how quickly the agent learns.

### 3.2 Stability and Multi-Seed Setup

To avoid drawing conclusions from single lucky or unlucky runs, we train each algorithm with multiple random seeds. For the main algorithm comparison (random vs REINFORCE vs PPO) we use three seeds per method and then compute, for each episode index, the mean and standard deviation of the reward across seeds.

Formally, if  $R_e^{(k)}$  denotes the reward in episode  $e$  for seed  $k$ , then the mean across  $K$  seeds is

$$\bar{R}_e = \frac{1}{K} \sum_{k=1}^K R_e^{(k)}, \quad (4)$$

and the standard deviation is

$$\sigma_e = \sqrt{\frac{1}{K} \sum_{k=1}^K \left( R_e^{(k)} - \bar{R}_e \right)^2}. \quad (5)$$

We plot  $\bar{R}_e$  as the main line and visualize  $\bar{R}_e \pm \sigma_e$  as a shaded region. This shaded band represents the spread we can expect when re-running the same configuration with another random seed. Narrow bands mean “reproducible”, wide bands mean “seed-dependent”.

For a separate stability experiment we run PPO with five different seeds (0–4) using the same default hyperparameters and again plot mean  $\pm$  standard deviation. This isolates the effect of randomness when everything else is kept fixed.

### 3.3 Hyperparameter Sensitivity Experiments

The main focus of the project is to see how sensitive PPO is to its hyperparameters. We choose three that are both important in theory and frequently tuned in practice:

- **Discount factor  $\gamma$ :** values 0.90, 0.99, 0.999.
- **Learning rate:** values  $3 \cdot 10^{-5}$ ,  $3 \cdot 10^{-4}$ ,  $3 \cdot 10^{-3}$ .
- **Clip range  $\epsilon$ :** values 0.1, 0.2, 0.4.

For the discount factor study we use three seeds per value (0, 1, 2). For each combination of  $\gamma$  and seed we train PPO and save the episode rewards. The final plot shows, for each  $\gamma$ , the mean curve and a shaded standard deviation band over seeds. This gives us a small “hyperparameter vs stability” picture.

For learning rate and clip range we focus on a single seed (seed 0) and vary only one hyperparameter at a time. This keeps the plots easy to read and already shows strong effects, even though in an ideal world each setting would also be averaged over multiple seeds.

## 4 Results

### 4.1 Random vs REINFORCE vs PPO

Figure 1 shows the comparison between the three agents when each is averaged over three seeds.

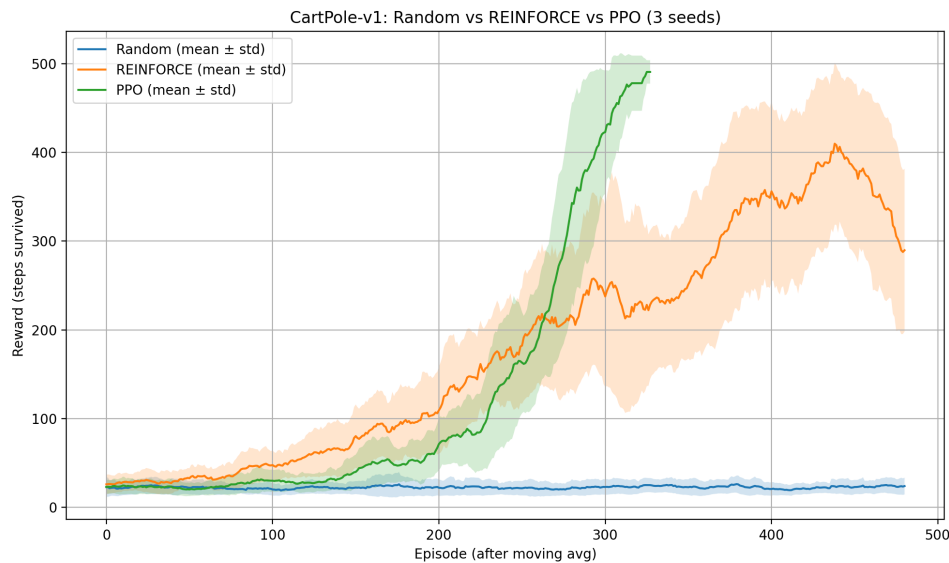


Figure 1: CartPole-v1 learning curves for random policy, REINFORCE, and PPO. Each solid line is the mean episode reward over three seeds; the shaded area shows one standard deviation above and below the mean. The random agent remains near 20–25 reward. Both REINFORCE and PPO eventually reach high returns, but PPO does so earlier and with a much narrower shaded band, indicating more consistent performance across seeds.

The random agent stays flat around a return of 20–25, as expected. REINFORCE and PPO both eventually reach high rewards, but PPO does it earlier and the shaded band is much narrower. After about 250 episodes, all PPO runs are close to the maximum score, while the REINFORCE runs still differ quite a bit from each other. This suggests that PPO is a better candidate for automated systems that cannot babysit each run.

To understand REINFORCE more closely, Figure 2 shows one single run.

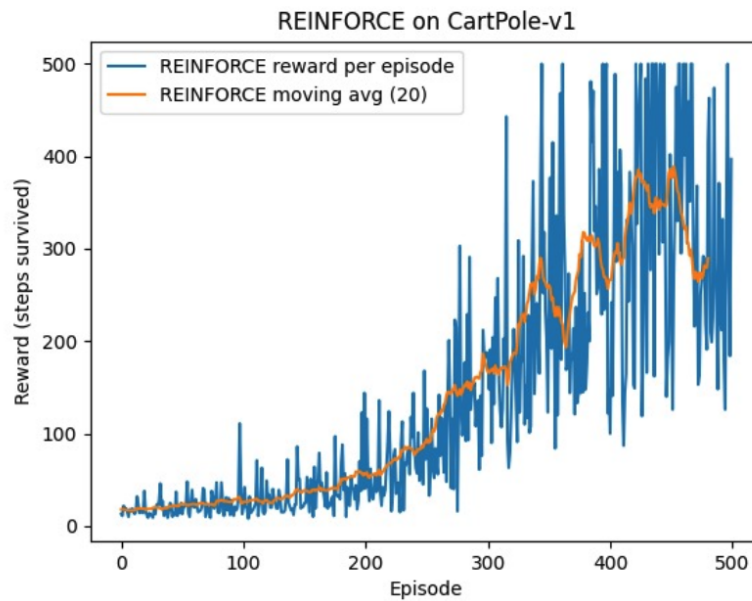


Figure 2: Single REINFORCE run on CartPole-v1. The blue curve shows raw episode rewards, while the orange curve is a moving average with window size 20. The large spikes and drops illustrate the high variance of the REINFORCE update rule: individual unlucky episodes can dramatically reduce performance before the policy recovers.

Training this agent feels unstable: the policy improves but occasionally collapses for several episodes before recovering. In contrast, PPO with default settings behaves much more predictably across seeds, as shown in Figure 3.

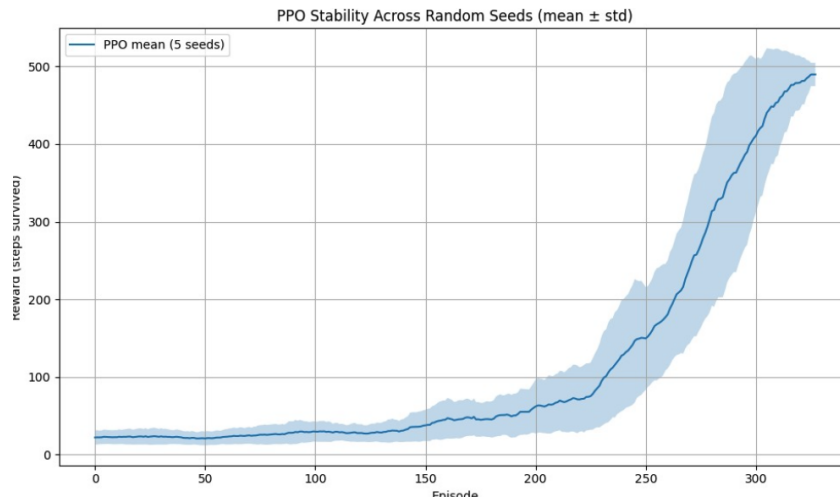


Figure 3: PPO trained with five different random seeds (0–4) using the default hyperparameters. The solid line shows the mean reward over seeds per episode, and the shaded band again corresponds to one standard deviation. After an initial warm-up phase the shaded area narrows, meaning that different seeds lead to very similar learning curves and PPO becomes highly stable.

Here the standard deviation shrinks over time and all seeds converge to very similar performance. This is exactly the type of behaviour we would like to see before placing an automated hyperparameter search on top.

## 4.2 Discount Factor Sensitivity

We now investigate how much the discount factor influences PPO on this supposedly easy task. Figure 4 shows the results when varying  $\gamma$  and averaging over three seeds per value.

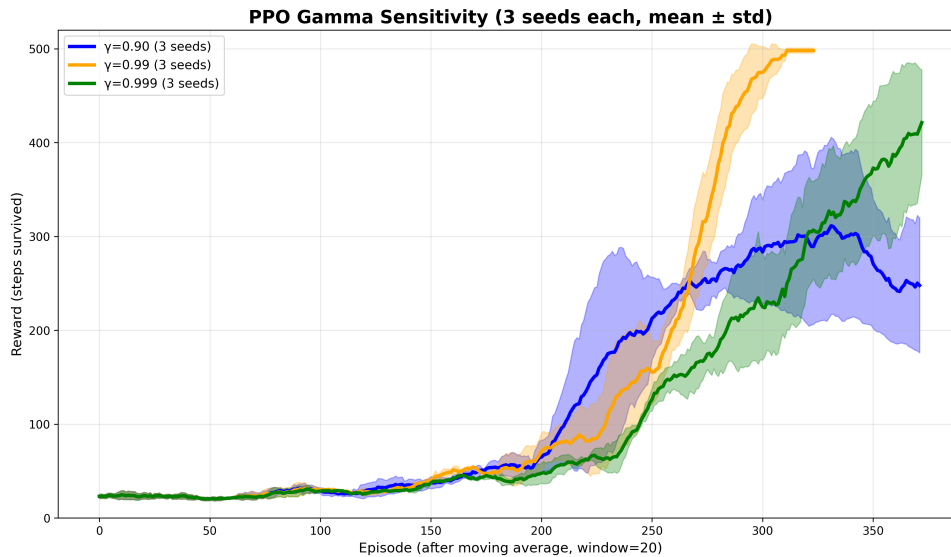


Figure 4: PPO discount-factor sensitivity with three seeds per value. For each  $\gamma \in \{0.90, 0.99, 0.999\}$  the solid curve is the mean reward over the three seeds and the shaded region is one standard deviation. With  $\gamma = 0.90$  (blue) learning improves initially but then plateaus around 200 reward, and the shaded band remains wide. With  $\gamma = 0.99$  (orange) PPO reaches the maximum reward quickly and the band becomes very narrow, indicating robust convergence. With  $\gamma = 0.999$  (green) the agent also succeeds but converges more slowly and shows more variability across seeds.

The result is quite striking. With  $\gamma = 0.90$  PPO usually learns some basic balancing but then stalls around 200 reward, even though the code and all other settings are identical. With  $\gamma = 0.99$  the agent reliably reaches the maximum score in roughly 150–170 episodes and the shaded band is small. With  $\gamma = 0.999$  the agent also solves the task, but it takes longer and the variance between seeds is larger.

From these experiments,  $\gamma = 0.99$  appears to be the best choice for this environment: it balances stability, speed, and robustness. At the same time, the difference between 0.99 and 0.90 shows how easily a reasonable-looking but slightly wrong value can make an otherwise strong algorithm appear to fail.

### 4.3 Learning Rate Sensitivity

Next we fix  $\gamma = 0.99$  and vary the learning rate. Figure 5 shows the behaviour for a single seed.

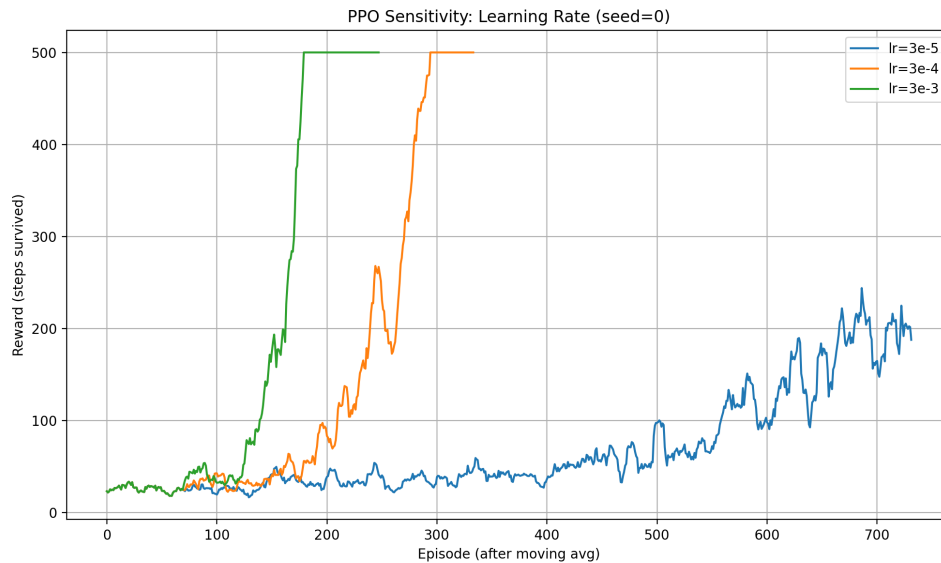


Figure 5: PPO sensitivity to the learning rate (single seed). A very small learning rate  $3 \cdot 10^{-5}$  (blue) leads to slow and noisy improvement and does not reach the maximum reward within the training budget. The default value  $3 \cdot 10^{-4}$  (orange) converges in a stable way and reaches 500 reward. A large learning rate  $3 \cdot 10^{-3}$  (green) learns very quickly but the curve shows sharper jumps, indicating that updates are close to the stability limit.

The pattern here is intuitive but still important. With a very small learning rate the agent improves only slowly and does not fully solve the task within the same number of timesteps. The default value  $3 \cdot 10^{-4}$  provides a good trade-off between speed and stability. The large learning rate  $3 \cdot 10^{-3}$  shoots up quickly but also overshoots and becomes more volatile. For automated tuning this means that simply sampling learning rates on a wide log-scale can easily hit both “too slow” and “too unstable” regions.

#### 4.4 Clip Range Sensitivity

Finally, we investigate the clip range  $\epsilon$  in the PPO objective. This parameter directly controls how far the policy is allowed to move in one step. Figure 6 shows three values for a single seed.

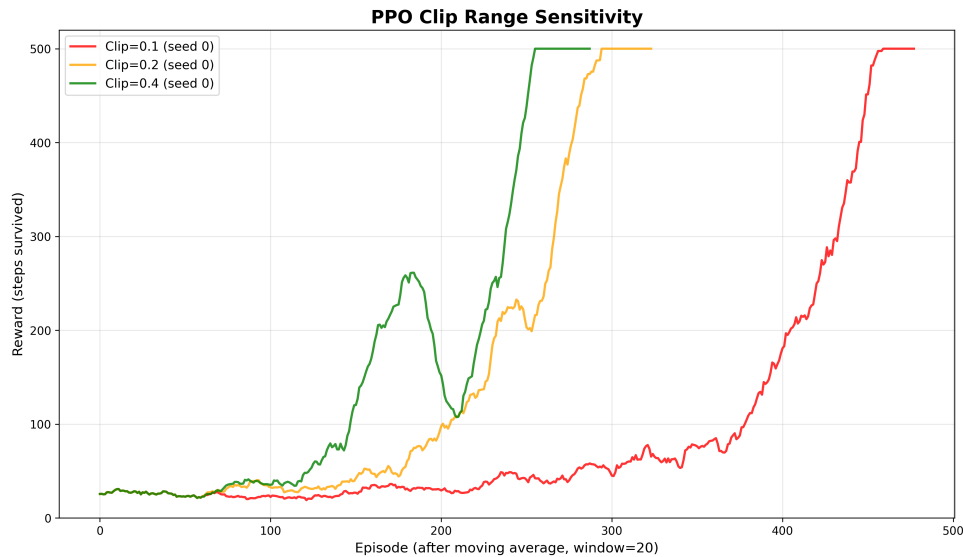


Figure 6: PPO sensitivity to the clip range (single seed). With a small clip range  $\epsilon = 0.1$  (red) the policy is updated very conservatively, so learning is slow and the agent only reaches high reward late in training. The default value  $\epsilon = 0.2$  (orange) produces fast and smooth convergence. With  $\epsilon = 0.4$  (green) updates are more aggressive and the agent reaches 500 reward very quickly, but the trajectory shows stronger oscillations, meaning the policy steps are almost too large.

A smaller clip range of 0.1 restricts updates so much that the agent learns only slowly and needs many episodes to get close to 500 reward. The default value 0.2 again looks like a good compromise between speed and stability. At 0.4 the agent learns very quickly but the curve is less smooth, which suggests that the theoretical stability guarantee of PPO becomes weaker. In combination with the discount-factor and learning-rate experiments, this again supports the view that there is no single magic setting; instead there is a fairly narrow region where things work nicely.

## 5 Discussion

Putting all experiments together, the picture is quite clear. First, PPO is a big practical improvement over REINFORCE on CartPole. It learns faster on average and, more importantly, is much more stable across different random seeds. The shaded regions in our plots quantitatively support this statement: PPO's standard-deviation bands become small after training, while REINFORCE keeps a wide band and large fluctuations. This makes PPO a reasonable base algorithm for AutoRL systems, because at least the underlying method is not constantly collapsing.

Second, even PPO is surprisingly sensitive to hyperparameters. Among the parameters we study, the discount factor  $\gamma$  has the largest qualitative effect: changing it from 0.99 to 0.90 can prevent the agent from ever solving the environment, despite identical code and a similar training budget. Learning rate and clip range also matter: too small values give

very slow learning, while too large values cause instability. For CartPole, our experiments suggest that

- $\gamma = 0.99$  is the most reliable discount factor,
- a learning rate around  $3 \cdot 10^{-4}$  is a good default,
- a clip range around 0.2 balances safety and speed.

These settings give fast learning and narrow shaded bands, which means both good performance and reproducibility.

For AutoRL this has two consequences. On the one hand, the existence of good hyperparameter settings means that automated search has something to discover. On the other hand, the high sensitivity means that naive manual tuning is unlikely to be sufficient, especially on larger and more expensive tasks. The results here align with the broader AutoRL argument that automated configuration and meta-learning will be necessary components of practical RL systems [4].

There are also limitations. All experiments are on a single, relatively easy benchmark. The number of seeds is still modest compared to what one would do in a large-scale study, and for learning rate and clip range only one seed per setting is used. Future work could extend this setup to continuous-control tasks, include more hyperparameters (batch size, entropy bonus, network architecture), and integrate a simple AutoRL method such as random search, Bayesian optimization, or population-based training on top.

## 6 Conclusion

This project started with a simple observation: running REINFORCE twice can give completely different curves. By building a small but systematic set of experiments on CartPole, we see that this is not just bad luck but an inherent property of many RL methods. PPO clearly helps with stability and reproducibility, especially when evaluated over multiple seeds, but it is still sensitive to key hyperparameters such as the discount factor, learning rate, and clip range.

Even on a toy environment, it is very easy to make a strong algorithm look bad by choosing slightly unfriendly hyperparameters. The shaded regions in our plots make this fragility visible: some configurations have narrow bands and clean convergence, others have wide bands or get stuck. This makes the case for AutoRL very concrete: if careful tuning is essential even for CartPole, then fully manual configuration on complex real-world tasks is simply not realistic.

## References

- [1] Peter Henderson et al. “Deep reinforcement learning that matters”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 32.1 (2018).
- [2] Ronald J. Williams. “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”. In: *Machine Learning* 8.3 (1992), pp. 229–256.

- [3] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [4] Jack Parker-Holder et al. “Automated Reinforcement Learning (AutoRL): A Survey and open problems”. In: *Journal of Artificial Intelligence Research* 74 (2022), pp. 517–568.
- [5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.